NAZARBAYEV INTELLECTUAL SCHOOL OF CHEMISTRY AND BIOLOGY IN KARAGANDA CITY

section: **AI programming**

« Using Artificial Intelligence in Music Writing»

Made by: **Abeuov Yerassyl,**
Grade 10F

Mentor: **Ustinova Lyudmila Valeryevna,**
Teacher of informatics

Karaganda 2025г.

Abstract
The project explores the possibility of using artificial intelligence in creating music. The rapid development and widespread use of neural networks and a significant increase in revenue in the music industry justify its relevance. The hypothesis is that neural networks can simplify the creation of musical compositions. The ultimate goal is a product capable of generating harmonious music using built-in neural networks. First, the topic of music and AI itself, analogs of the future program are studied, a programming language and libraries are selected, and then the program code itself is written. All research and creation was carried out independently by the author of the project. As a result, the use of neural networks significantly facilitated the creation of a music generator, the hypothesis is confirmed. The program is suitable for use, it can be used by sound designers of works of art, composers, etc. It is planned to release to the general public.

Introduction

Music is an integral part of humanity. This type of art can embody various artistic images and convey all sorts of human emotions through the harmonious organization of certain sounds. It can be performed both from improvised instruments or voice, and with numerous musical instruments. Nowadays, music has long become an entire industry, a business where billions of dollars are circulating. The total revenue of the music industry in 2023 was about 28.6 billion US dollars, which is 10% more than in the previous year, while the revenue in 2014 was only 13 billion US dollars [1]. This indicates that the music business is developing every year. Artificial intelligence technologies are also developing rapidly. Artificial intelligence (abbreviated AI) is a universal term for technologies that create a program that imitates human qualities. Its use is aimed at facilitating certain tasks in various fields. AI is one of the most discussed topics of the 21st century due to its growing popularity and rapid development. Machine intelligence is already widely used in industry, mechanical engineering, predictive analytics, and the service sector. AI is also widely used in writing music.
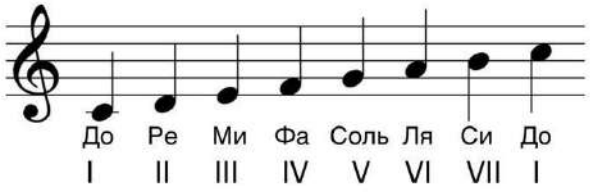
The history of "software" music begins in 1951, when Alan Turing, a British mathematician, first created a computer-generated piece of music. The "Turing Machine" that created this song, and which Alan himself built, filled most of the first floor of the Manchester Computer Laboratory [2]. Now there are a large number of websites and computer programs that can generate a melody in a matter of seconds using a built-in neural network. The relevance of AI music (music generated by artificial intelligence) is that most songs written by humans are protected by copyright and are prohibited for commercial use. The difficulty of finding or creating background music for advertising without copyright creates problems for businesses. In addition, sound designers responsible for writing the musical component of films or video games, and composers, thanks to neural networks, have the opportunity to take inspiration from AI to write their own musical works. In this project, I plan to create a computer program whose built-in neural network is capable of generating melodies, namely a chord sequence depending on the mood specified by the user and the first chord of the sequence, also adding drums. It should be easy to use. I have set myself several tasks, upon completion of which the desired product should be created. The first task is to familiarize myself with music theory for its future implementation in the code, which should give greater harmony to the generated notes. It will be necessary to familiarize yourself with how chords are composed for a particular key. Also, you need to get acquainted with the work of chord progressions in minor and major. In addition, the first task is also aimed at familiarizing yourself with the work of neural networks. The second task is a review of analogs of my computer program. The implementation of this task will allow me to identify the shortcomings of analogs, which, if possible, should be taken into account in my final code. The third task is to plan the future program code, in particular its structure. This will help me choose a programming language and libraries more effectively and easily. The fourth task is to analyze popular programming languages and determine the most effective and convenient language for writing my code. Comparing the advantages and disadvantages of certain languages will help with this task. The fifth task is to find and analyze libraries that can implement my idea. Libraries should be compatible with the selected programming language and provide convenient functions and tools for implementing my program. The sixth task is to write the main program code. After completing all the tasks, the last stage of the project will be the conclusion with the analysis and assessment of all the work done. The final goal of my project is to create unique software (the purpose of which was discussed earlier), the potential consumers of which may be

musicians who can use the generated chord sequence when creating musical accompaniment to their songs, games, advertisements, etc.
Problems in implementing my idea may be the complexity of the required program code, research and analysis of information on the creation and training of neural networks mainly in a foreign language (due to the lack of a sufficient amount of required information in Russian), the study of musical theory and its implementation in the code.

Research section

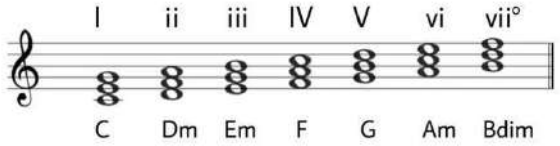
The first task. Introduction to the theory.

Music has a certain scientific theory that musicians study and use in practice. It explains the harmony of certain sounds played together or in sequence. Professional musicians quite often use chords in their works. Chords are a combination of several different notes (3 or more) that are played simultaneously. There are different types of chords that can convey different moods: minor chords are considered melancholic, major chords are considered positive. In addition to them, there are quite a large number of other chords with different emotions, but it is minor and major chords that are the most basic for creating a quality piece of music. A quality musical composition often has a chord sequence based on harmonic logic (alternative name - chord progression), suitable for the key of the song itself. Tonality in music is a certain organization of sounds (notes), in which the tonic (the main note) serves as the central and defining one for the work. Each key consists of seven steps, which are notes. Let's look at this using the key of C major as an example.



Picture 1. C Major scale.

C major has seven steps, alternating every octave (the interval between the nearest identical sounds of different pitches). Each step is numbered with Roman numerals. The notes of the steps vary depending on the mode and key. In each mode, there is a certain distance between the pitches of the notes. These distances set the notes of the steps for a certain key, which are based on the tonic (the first step of the key), from which the distances of other steps are measured. When writing a composition that is pleasant to the human ear, the notes of the key of the song are most often used to prevent dissonance. Each key has seven chords, the tonics (main notes) of which are the notes of the key. Let's look at the example of C major.

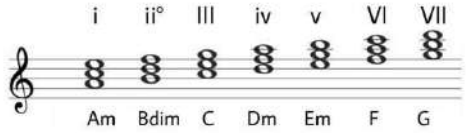


Picture 2. C major chords.

As you can see in Picture 2, chords are also designated by Roman numerals, with lowercase numbers meaning a minor chord, capital numbers meaning a major chord, and numbers with a small circle designation meaning a diminished chord. The sequence of these Roman numerals when composing chords is present in all major keys. For example, the second degree chord of D major will also be minor.

These chords are divided into three types. Chords of the first and third degrees are often called tonic, although the third is more correctly called mediant. Chords of the second, fourth and sixth

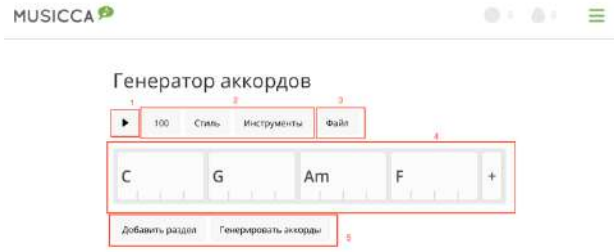
degrees are often designated as subdominant, fifth and seventh as dominant. Without going too deep into musical theory, we can note the rule used in composing chord sequences - from tonic chords you can go to subdominant or dominant, after subdominant chords follow dominant, after dominant follow tonic. Let's consider the same thing in the key of A minor natural.



Picture 3. A minor chords..

The Roman numeral sequence in Figure 3 is present in all minor keys when composing chords. The chord types (major, minor, diminished) of the degrees have changed due to the different distances between the degrees. Despite this, the rule also applies here, on certain degrees the same chord types (subdominant, tonic, dominant) are used.
The Roman numeral notation is quite convenient for composing chord sequences, which can then be easily transposed (changed to a different key). With the help of Roman numerals, you can compose an infinite number of chord progressions in different keys.
The reason for generating chords in my program is that you can write a sequence of notes in a song based on chords. In other words, it is easier to match notes to chords than to match chords to notes. Notes can be roughly "sung", while composing chords requires an in-depth knowledge of music theory. Since my product is going to use neural networks, I need to get to know the general concept of artificial intelligence better before doing so. Artificial intelligence is a field of computer science devoted to creating systems and programs that can perform tasks that usually require human intelligence. Neural networks (abbreviated as neural networks) are computing systems consisting of interconnected artificial neurons that work together to solve problems. They learn from training data by adjusting the parameters of the model in such a way as to minimize the error in predictions. In other words, a neural network is trained on certain data, looking for a possible certain pattern in it, thanks to which the neural network can predict or recognize something in practice. This certain data will need to be identified and created during the course of the project.

The second task. Analogues.
There are analogs of my chord progression generation program. There are programs in both Russian and English.



Picture 4. Musicca.com

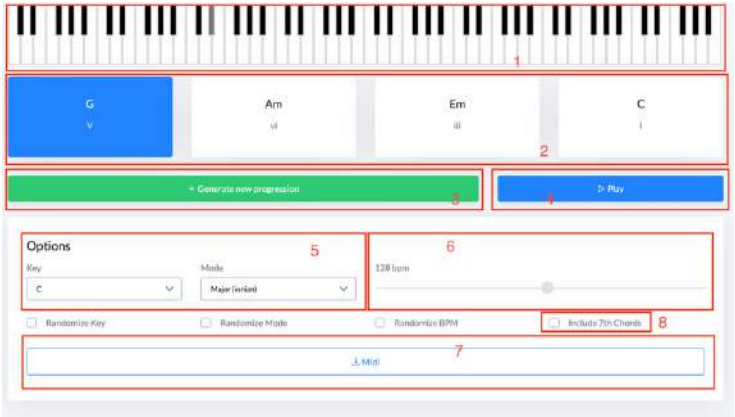
The only popular Russian-language analogue is the “Chord Generator” tool on the “Musicca.com” website. This free tool allows the user to generate a chord sequence depending on the style selected by the user (block 2). The program creates a composition based on the

generated progression (it was created when you clicked “Generate Chords” in block 5), the selected BPM and style (block 2), where the progression itself can be played when you click a button (block 1) with accompaniment (drums, piano, bass, guitar). You can add and change chords, change their duration (how many bars a particular chord is repeated) (block 4), turn on or off the playback of certain instruments (block 2). It is saved only by the registered user (you can save it in block 3) exclusively on the site itself, i.e. you cannot download the audio file of the generated song. In addition, the program generates only according to clearly defined rules and previously recorded progressions, and it cannot predict the next added chord itself, because it does not have a trained AI model (a programmed model with artificial intelligence) for this.



Picture 5. AutoChords.com

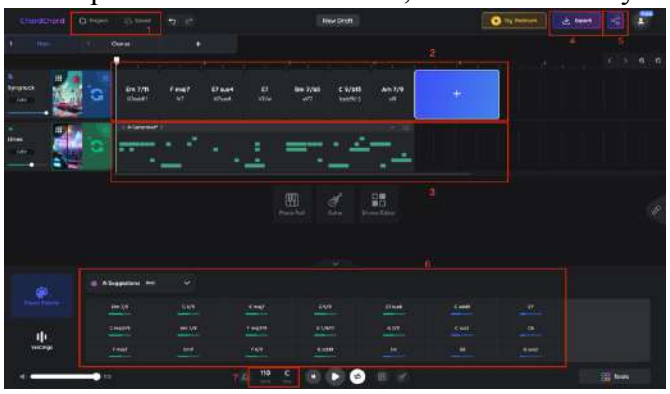
There are many more English-language analogues on the Internet. It is worth highlighting the free site “Autochords.com”, which can generate sequences depending on the selected scale, key and “name” parameter (they can be selected in block 1), which can be designated by a certain mood of the progression. This parameter has the options “Cliché”, “Energetic”, “Memories”, “Simple” and many others. Chords are played using a specified instrument (guitar or piano, you can select in “Instrument” in block 1). You can see what notes are in a certain chord (by clicking on the chords in block 2). The disadvantage of the program is the lack of output of the progression to an audio file. Another disadvantage is the uniformity. For one “name”, key and scale, there are only four specific sequences.



Picture 6. soundgrail.com.

The chord progression generator tool on the Soundgrail.com website can do almost the same thing as the previous English-language analogue, except for the lack of a mood selection. The chords (in block 2), with the same settings (they are configured in block 5), change with each new generation (generation is carried out by pressing the button in block 3). There is a button for adding seventh chords (unlike simple chords, they consist of four sounds) to the progression (block 8). When playing the progression (by pressing the button in block 4), the chords are

played on the piano (block 1). It is possible to select the BPM of the chords (block 6) and output the progression to a MIDI file (the output button in block 7). The MIDI file contains information about the sequence and duration of the notes, and it is used mainly by professional beatmakers in specialized software for creating music. However, to withdraw, you need to purchase a subscription on this site, the monthly cost of which is 6 US dollars.



Picture 7. ChordChord.

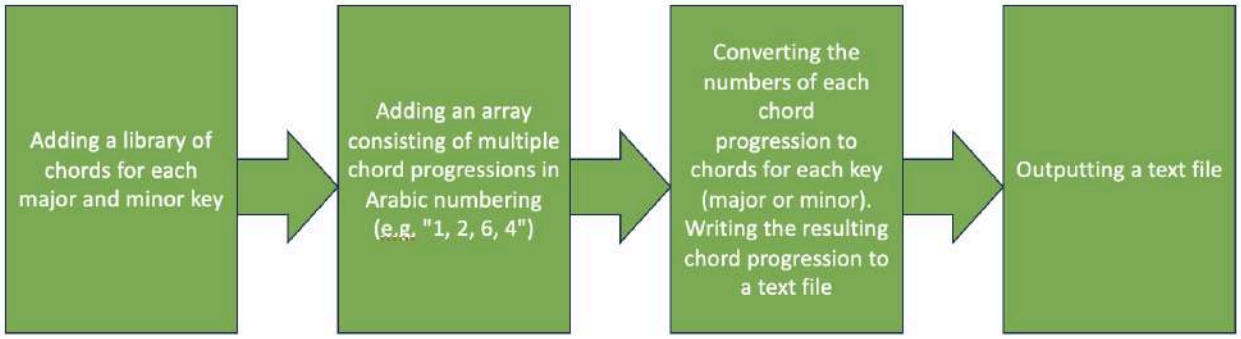
One of the most popular and high-quality chord sequence generators is “ChordChord”, which specializes in this task. The user can either select the song genre, and the built-in AI will generate the chords itself, or start everything with an empty project. The program is similar to a DAW (music editor), where the user can select the key, scale, BPM of the composition in the project (in block 7) and create a simple accompaniment (let's say, a MIDI drum track in block 3). When adding and changing chords (in block 2), the user can use the “AI Suggestions” tool (block 6), where the neural network will offer suitable and harmoniously sounding chords, also evaluating their “harmony”. The created project can be saved in your profile (in block 1, you need to register to save). This project can be shared with another user by sending an invitation link (the “Share” button in block 5). To export a composition to MIDI, WAV, MP3, PDF (its button in block 4) and to use other functions of the program, you need to purchase one of two subscriptions. Their monthly cost is 12 and 27 US dollars, annual - 99 and 147 US dollars. The difference between the subscriptions is that the cheaper one has its own limitations in export, generation and AI suggestions, while the expensive one does not have them.

There are few Russian-language analogues of my program. Unlike my program, they do not use artificial intelligence. Export of progressions is a paid function in all popular English-language analogues. The free version of the decent program “ChordChord” is limited, and the program itself does not support the Russian language.

The third task. Planning the program code.

Planning the program code is one of the key stages of my work. Having dealt with the plan, it is much easier to choose a programming language and libraries. My program should request the mood of the song, the key and the bpm (it measures the speed of the musical composition), then generate a chord progression according to this user choice. The code should contain an AI model trained to build progressions. It should be trained on the example of different chord sequences written in a line in text files “.txt” (one for minor, one for major). The advantage of this format is its very small size, which will speed up the training of the AI model. The file should have at least several hundred progressions in all keys, which we will have to output separately from the main

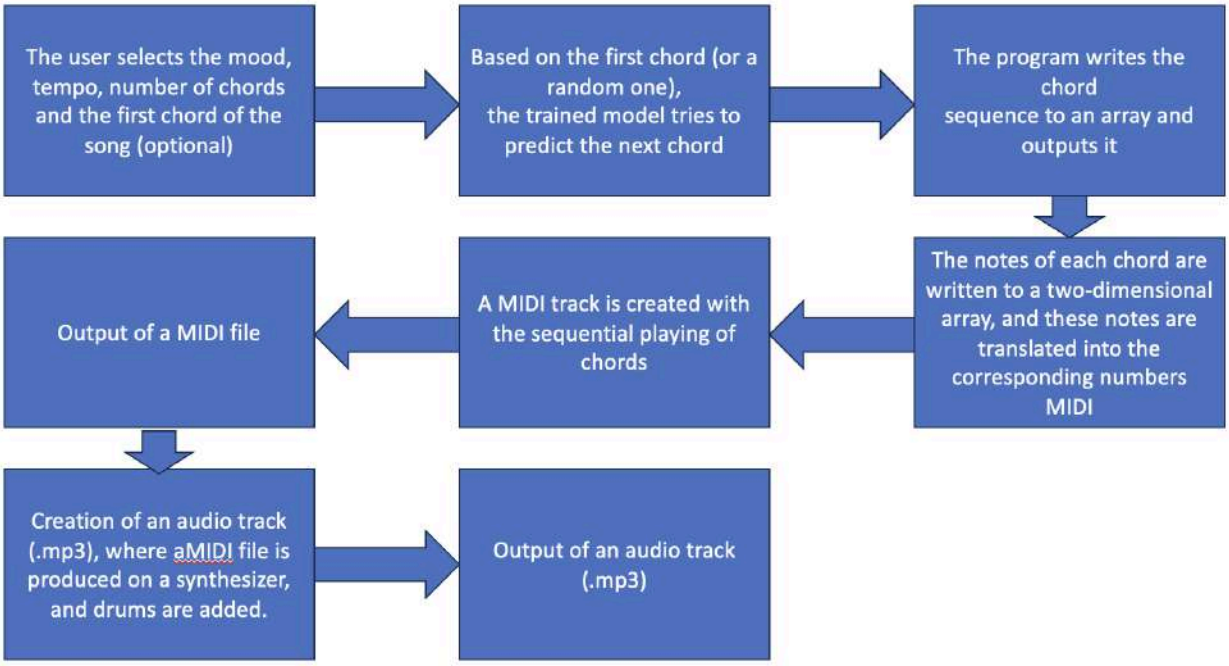
code. The additional code should have chords for each major and minor key, which can be written in certain arrays. It should also have an array filled with chord sequences written in Arabic numbering. The program substitutes Arabic numbering for each key and outputs chords in Latin form. We will write the result for minor and major in two different text files.



Picture 8. The scheme of the additional code.

Then, we train the AI model to sequence chords and create a function where it returns the generated progression as an array. Training the model will require creating another additional code that outputs a file with the trained neural network. The main code should have functions that create a two-dimensional array consisting of the notes of each generated chord in each subarray. These notes are translated into specific MIDI numbers for subsequent creation of a MIDI file. Then, using this file and a specific library, an audio track is output, where MIDI notes are played on a synthesizer at a given tempo. As stated earlier, the sequence must be output verbally, in a MIDI file (.mid) and in an audio file (.mp3). Examples of such software are Ableton Live, FL Studio, Logic Pro X - world-famous applications for writing musical compositions. In such software, you can change the speed of playback of notes, the instruments on which a particular MIDI track is played. This means that this file type is convenient for further work with the generated music, as well as for changing it. Output in mp3 format provides the user with a more simplified perception of the generated chord sequence - the audio track contains the sound of a certain synthesizer playing the generated progression, say 2-3 times. The MP3 format has many advantages. Firstly, the small file size due to compression provides less load on the device using the program. Secondly, support for this format by almost all popular

computers allows my program to be available to a larger number of people.



Picture 9. The scheme of the main code.

A unique feature of my product will be the interface in Russian and the use of neural networks, which was not available in Russian-language and many English-language analogues. Their use will reduce the time for implementing additional music theory, which the AI will be able to learn on its own.

The fourth task. Choosing a programming language.

The Stack Overflow developer survey for 2023[3] showed that the most popular programming languages are JavaScript, HTML/CSS, Python, and SQL. Since I plan to create a computer program focused on Windows, the most popular operating system for computers, languages such as Kotlin, used to develop applications for Android devices, and Swift, used to develop programs for Apple devices, are not needed for my project. SQL is used to work with databases, which is not related to the tasks of my program. HTML/CSS is used to develop websites, which I do not plan to do.

The choice remains between JavaScript and Python. During the study of their advantages, this table was compiled with the advantages of each language and their similar features.

Python	Together	JavaScript

• Often used for scientific computing, data analysis • Simplicity and "flexibility" of code • Many amateur ratings indicate it as the best programming language for working with artificial intelligence	• Large number of libraries • Large community of programmers	• Used in web development, often being the primary language for front-end development • Convenient for creating interactive user interfaces

Table 1. Comparison between Python and JavaScript.

Being a fairly lightweight and flexible language suitable for working with neural networks, Python is best suited for implementing my idea, while JavaScript is more focused on web development.

Task 5. Analysis and selection of libraries.

Python has many libraries for working with artificial intelligence, MIDI, chords, matrices, and other tasks. I got to know the most popular ones while researching various websites about Python programming.

To create an AI model and train it, I will use the TensorFlow library (along with Keras). This is a popular machine learning library that allows you to develop and train complex models. Keras is a high-level API for creating and training AI models, built into TensorFlow, offering easy-to-use tools for rapid prototyping and research.

To work with MIDI, the following libraries will be used: os, pydub, midiutil. MidiUtil is capable of creating MIDI files based on notes recorded in MIDI numbering. The "os" library, which is the standard Python library for interacting with the operating system, is needed to call FluidSynth to convert the MIDI file to ".wav". FluidSynth, which you will need to install in advance, is a software synthesizer that converts MIDI data to audio. It uses SoundFont audio fonts (".sf2" or ".sf3") to generate audio. The "AudioSegment" module from "pydub" will be used to convert the ".wav" file to ".mp3", since "FluidSynth" needs ".wav" to work.

The "os" module in Python is also needed for the code to interact with the operating system, including working with files and managing directories, thereby allowing you to perform various operations related to the file system. Roughly speaking, the code will understand where this or that imported file is located.

The "Chord" module from the "pychord" library will be used, thanks to which it will be possible to display information about the chord.

The NumPy library, which is the main library for scientific computing in Python, has a set of mathematical functions for array operations. This will be needed in the code when training AI models using text training files.

To create the graphical interface of the program, the tkinter library will be imported. It provides tools for creating windows, buttons, text fields, and other interface elements. Together with it, its filedialog module will be imported. The module provides dialog boxes for selecting files or directories. It allows the user to select the location to save files (in the case of my program, these are MIDI and MP3 files).

Some Python modules will also be used: pickle (for saving and loading AI models and dictionaries), random (random selection of the initial chord at the user's discretion), tempfile (for

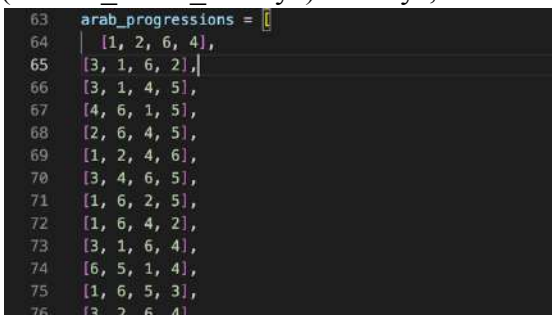
creating temporary files, if, for example, the user wants to immediately output the file to wav, which will require creating a MIDI file of the chord progression).
All libraries were found by independently searching the Internet.

The sixth task. Writing the code.
To obtain a text file that will be a dataset for training the main code AI model, we need to write additional code.



Code part 1.

At the beginning of the code, I wrote two arrays for major (“major_chord_library”) and minor (“minor_chord_library”) keys, each key has its own list of chords.



Code part 2.

After this, there is another one-dimensional array “arab_progressions”, consisting of many chord progressions in Arabic numeration. All arrays are filled in manually. It is worth noting that music theory was used when filling them in. Roman numerals are not used because the register and number symbols denote the type of chords, but we already have everything written in advance in the two previous arrays.

```
def arab_to_real_chords_in_all_keys(input_progression, lad):
    all_chord_progressions = []

    if lad.lower() == 'major':
        chord_libraries = major_chord_library
    elif lad.lower() == 'minor':
        chord_libraries = minor_chord_library
    else:
        return "Invalid mode. Please choose 'major' or 'minor'."

    for note in chord_libraries:
        chord_list = chord_libraries[note]
        chords = []

        for num in input_progression:
            chord = chord_list[num - 1]
            chords.append(chord)

        all_chord_progressions.append(chords)

    return all_chord_progressions

```

Code part 3.

The main function of the additional code “arab_to_real_chords_in_all_keys” (translates as Arabic [numbers] to real chords in all keys) has the parameters “input_progression” (any chord progression in Roman numerals) and “lad” (the mode, major or minor). To begin with, the function creates an empty list “all_chord_progressions”, which will contain all chord progressions for all keys. Depending on the given mode, the corresponding chord library is selected: either “major_chord_library” for the major mode, or “minor_chord_library” for the minor one. If the entered mode is invalid, the function will return an error message. The loop “for note in chord_libraries” iterates through each note in the chord library (for all possible keys). "Chord_list" is set to the list of chords for the current note in the current key. In the "for num in input_progression" loop, for each number in "input_progression" the corresponding chord in "chord_list" is found using that number as an index. For example, if "num" is 1, the first chord in "chord_list" is selected. At the end of the first loop, the list of chords for the current key is added to the list of all chord progressions. When the function completes, a list of all chord progressions for all possible keys is returned.

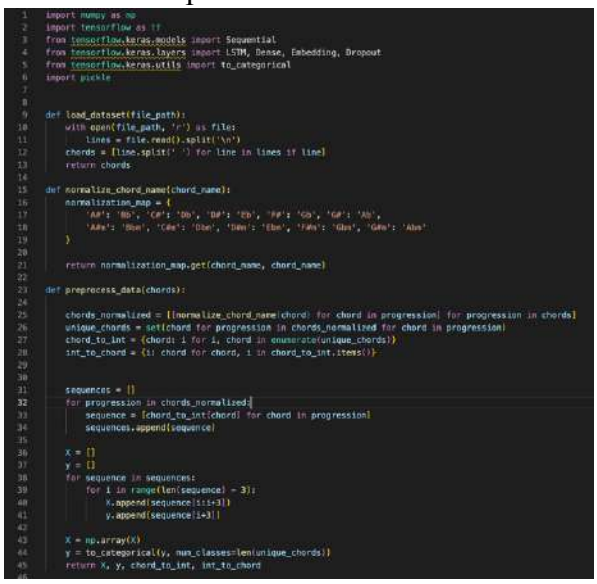
```
lad = 'major'
with open('happy_chord_progressions.txt', 'w') as file:
    for progression in arab_progressions:
        output = arab_to_real_chords_in_all_keys(progression, lad)
        for chords in output:
            file.write(' '.join(chords) + '\n')
```

Code part 4.

After that, the "lad" mode (major or minor) is specified and the "chord_progressions.txt" file is opened for writing. The code then loops through each roman numeral progression, converts it to chords for all keys using the "arab_to_real_chords_in_all_keys" function and writes the result to the "chord_progressions.txt" file. Each chord progression is written in a separate line of the file.

Let's create two files using this code - "major_chord_progressions.txt" (which is created for the major scale) and "minor_chord_progressions.txt" (which is created for the minor scale).

Also, two codes are needed to train two AI models with a dataset of sad and happy progressions. The code template that will work for both models is an implementation of a neural network.



```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Embedding, Dropout
from tensorflow.keras.utils import to_categorical
import pickle


def load_dataset(file_path):
    with open(file_path, 'r') as file:
        lines = file.read().split('\n')
    chords = [line.split(' ') for line in lines if line]
    return chords

def normalize_chord_name(chord_name):
    normalization_map = {
        'A#': 'Bb', 'C#': 'Db', 'D#': 'Eb', 'F#': 'Gb', 'G#': 'Ab',
        'A#m': 'Bbm', 'C#m': 'Dbm', 'D#m': 'Ebm', 'F#m': 'Gbm', 'G#m': 'Abm'
    }

    return normalization_map.get(chord_name, chord_name)

def preprocess_data(chords):

    chords_normalized = [[normalize_chord_name(chord) for chord in progression] for progression in chords]
    unique_chords = set(chord for progression in chords_normalized for chord in progression)
    chord_to_int = {chord: i for i, chord in enumerate(unique_chords)}
    int_to_chord = {i: chord for chord, i in chord_to_int.items()}


    sequences = []
    for progression in chords_normalized:
        sequence = [chord_to_int[chord] for chord in progression]
        sequences.append(sequence)

    X = []
    y = []
    for sequence in sequences:
        for i in range(len(sequence) - 3):
            X.append(sequence[i:i+3])
            y.append(sequence[i+3])

    X = np.array(X)
    y = to_categorical(y, num_classes=len(unique_chords))
    return X, y, chord_to_int, int_to_chord
```

Code part 5.

The code uses several libraries that were mentioned earlier. Some modules from “tensorflow.keras” were also used. The “Sequential” module is used to build the model sequentially, i.e. layer by layer. “LSTM” (long short-term memory), “Dense” (fully connected layer), “Embedding” (layer for working with vector representations), “Dropout” (layer used for regularization to avoid overfitting) are used to build the neural network efficiently. “to_categorical” from “tensorflow.keras.utils” is a function to convert numeric labels into a binary class matrix.

The “load_dataset” function takes a file path as input (“file_path”), reads the file contents and splits them into lines, writing them to the “lines” array. After that, a list of lists (“chords”) is created, where each inner list contains the chords of one musical sequence. The list is returned.

The “normalize_chord_name” function normalizes chord names to a common format. It maps equivalent chord names (e.g. 'A#' to 'Bb', 'C#' to 'Db', etc.), returning the normalized chord name at the end. “chord_name” is the name of the chord to normalize. “normalization_map” is a dictionary for converting chords to their alternative names.

The “preprocess_data(chords)” function pre-normalizes chord names using the “normalize_chord_name” function, writing the results to “chords_normalized”. It then maps each unique normalized chord to an integer index (which it specifies itself) and vice versa, successively converting chord progressions into sequences of chord indices. “chord_to_int” and “int_to_chord” are dictionaries mapping chord names to unique numeric identifiers and back, “sequences” is an array of numeric representations of musical sequences. After normalizing the chord names and converting them to numeric indices, “X” and “y” are formed from the chord sequences. “X” is the input to the model. For each chord sequence, subsequences are created, each containing N chords (in this case, N is three). These subsequences are used as input to the

model, that is, they represent the context based on which the model will predict the next chord. “y” is the target value for the model. For each subsequence in “X”, the corresponding target value “y” is the chord that immediately follows this subsequence in the original musical sequence. That is, “y” contains the chord that the model should learn to predict given the chords that came before it. The function converts X into a numpy array suitable for training a neural network. In the end, “X”, “y”, “chord_to_int”, “int_to_chord” are returned.

```
46
47  def create_model(num_unique_chords):
48      model = Sequential([
49          Embedding(input_dim=num_unique_chords, output_dim=100, input_length=3),
50          LSTM(100, return_sequences=True),
51          Dropout(0.2),
52          LSTM(100),
53          Dense(num_unique_chords, activation='softmax')
54      ])
55      model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
56      return model
57
58  def train_and_save_model(file_path, model_name):
59      chords = load_dataset(file_path)
60      X, y, chord_to_int, int_to_chord = preprocess_data(chords)
61      model = create_model(num_unique_chords=len(chord_to_int))
62      model.fit(X, y, epochs=80, verbose=1)
63      model.save(f'{model_name}.h5')
64      with open(f'{model_name}_chord_to_int.pkl', 'wb') as f:
65          pickle.dump(chord_to_int, f)
66      with open(f'{model_name}_int_to_chord.pkl', 'wb') as f:
67          pickle.dump(int_to_chord, f)
68
69
70
71  train_and_save_model('/Users/yerassylabeuov/Desktop/untitled folder 2/happy_chord_progressions.txt', 'happy_model')
72
73
74
```

Code part 6.

The function “create_model(num_unique_chords)” returns the compiled model. It defines a sequential neural network. “model” is a sequential model containing certain layers. The “Embedding” layer transforms integer indices into dense vectors of fixed size. Two “LSTM” layers provide long-term and short-term memory to capture sequential dependencies. The “Dropout” layer is a regularization technique to prevent overfitting. A fully connected layer (“Dense”) with “softmax” activation is the output layer to predict the probability distribution over different chords. Then, the model is compiled with the “adam” optimizer and the “categorical_crossentropy” loss function, and the compiled model is returned.

The parameters of the “train_and_save_model(file_path, model_name)” function are “file_path” (path to the data file) and “model_name” (name to save the model). The function loads the dataset using the “load_dataset()” function, preprocesses the data using the “preprocess_data()” function, creates a model using the “create_model()” function and trains it on the preprocessed data. After training, the trained model is saved. Along with it, “chord_to_int” and “int_to_chord” are saved to disk using pickle. In two “with open(f'{model_name}_….pkl', 'wb')” (the ellipsis can be “chord_to_int” or “int_to_chord”) opens the file for writing in binary mode (“wb”). The file name is formed from the model name “model_name” with “{model_name}_….pkl” appended to indicate the file content. The “.pkl” extension is usually used for files saved with “pickle”. “pickle.dump(…, f)” writes the serialized dictionary to the opened file. At the end of the entire code, this function is called with the path to the dataset and its name.

Using this code and the previous text files, two models and two dictionaries for each model were created (4 dictionaries in total) — “happy_model.h5” (AI model of major progressions), “sad_model.h5” (AI model of minor progressions), “happy_model_chord_to_int.pkl” (dictionary of the major model displaying chords in numbers), “happy_model_int_to_chord.pkl” (dictionary of the major model displaying numbers in chords), “sad_model_chord_to_int.pkl” (dictionary of the minor model displaying chords in numbers), “sad_model_int_to_chord.pkl” (dictionary of the minor model displaying numbers in chords).

Now with the models and dictionaries it is possible to write the main code of the program.

```
import numpy as np
import tensorflow as tf
import pickle
from tkinter import *
from tkinter import filedialog
import os
import random
from pydub import AudioSegment
from pychord import Chord
from midiutil import MIDIFile
import tempfile
```

Code part 7..

The library data, the purpose of which was discussed earlier, is imported. FluidSynth also needs to be installed.

```
def load_model_and_dictionaries(model_name):

    model = tf.keras.models.load_model(f'Дополнительные_файлы/{model_name}.h5')

    with open(f'Дополнительные_файлы/{model_name}_chord_to_int.pkl', 'rb') as f:
        chord_to_int = pickle.load(f)
    with open(f'Дополнительные_файлы/{model_name}_int_to_chord.pkl', 'rb') as f:
        int_to_chord = pickle.load(f)
```

Code part 8.

The “load_model_and_dictionaries” function with the “model_name” parameter loads the AI model using the “tf.keras.models.load_model()” command, then loading the model libraries using “with open(…) as f” and “pickle.load(f)”. The loaded model is returned, as well as the chord and number dictionaries. The dictionary and model files are pre-added to the “Additional Files” folder, which is located in the folder with the main code.

```
options_without_any = ['C', 'Db', 'D', 'Eb', 'E',
                       'F', 'Gb', 'G', 'Ab', 'A',
                       'Bb', 'B', 'Cm', 'Dbm', 'Dm',
                       'Ebm', 'Em', 'Fm', 'Gbm', 'Gm',
                       'Abm', 'Am', 'Bbm', 'Bm']

def generate_chords(mood, start_chord, num_chords=3):

    model_name = f"{mood}_model"
    model, chord_to_int, int_to_chord = load_model_and_dictionaries(model_name)

    if start_chord not in chord_to_int:
        start_chord = random.choice(options_without_any)

    sequence = [chord_to_int[start_chord]]
    generated_chords = [start_chord]

    for _ in range(num_chords):
        x_pred = np.array(sequence[-3:])[np.newaxis, :]
        x_pred = np.pad(x_pred, ((0, 0), (max(0, 3-x_pred.shape[1]), 0)), mode='constant', constant_values=0)
        prediction = model.predict(x_pred, verbose=0)[0]

        next_index = np.random.choice(range(len(prediction)), p=prediction)
        generated_chords.append(int_to_chord[next_index])
        sequence.append(next_index)

    return generated_chords
```

Code part 9.

The “options_without_any” array is written with all possible simple major and minor chords. The “generate_chords” function with the parameters “mood” (str), “start_chord” (starting chord, str), “num_chords” (the number of chords in the progression, equal to three) is used to generate a chord sequence based on a given mood and a starting chord. The line “model_name = f"{mood}_model" forms the model name based on the “mood” value. Then the “load_model_and_dictionaries” function is called, which loads the corresponding model and two

dictionaries. If the “start_chord” value is not found in the “chord_to_int” dictionary, then a chord is randomly selected from the “options_without_any” list. The sequence list is initialized with the index of the starting chord. The “generated_chords” list is initialized with the starting chord. New chords are generated in a loop that is executed “num_chords” times. The slice of the last three elements of “sequence” is converted into an array ‘x_pred’, which in turn is fed to the model input. If the slice length is less than three, it is padded with zeros. The model makes a prediction based on “x_pred”, returning an array of probabilities of the next chord. The next chord is chosen randomly based on the obtained probabilities, and its index is added to “sequence”, and the chord itself is added to “generated_chords”. The function returns a list generated_chords containing the initial chord and the generated chords.

```
def chords_to_notes(chord_progression):
    notes_of_chord_progression = []
    for chord in chord_progression:
        c = Chord(chord)
        notes_of_chord_progression.append(c.components())
    return notes_of_chord_progression

def notes_to_midi_with_bass(chord_notes):
    note_mapping = {
        "C": 60, "Db": 61, "C#": 61, "D": 62, "Eb": 63, "D#": 63, "E": 64,
        "F": 65, "Gb": 66, "F#": 66, "G": 67, "Ab": 68, "G#": 68, "A": 69,
        "Bb": 70, "A#": 70, "B": 71, "Cb": 71, "Fb": 64
    }

    midi_chords = []
    bass_notes = []
    for chord in chord_notes:
        midi_chord = [note_mapping[note] for note in chord]
        midi_chords.append(midi_chord)
        bass_notes.append(note_mapping[chord[0]] - 12)
    return midi_chords, bass_notes
```

Code part 10.

The “chords_to_notes” function takes a list of chords and returns a list containing the notes of each chord. The “chord_progression” parameter is a list of strings, where each string is a chord. First, an empty “notes_of_chord_progression” list is created to hold the notes of each chord. Then there is a loop where for each chord in the “chord_progression” list, a “Chord” object is created, representing the chord itself. This object is needed so that, using the “components()” method, it is possible to extract the notes of the chord, which are then added to the “notes_of_chord_progression” list. This list is returned.

“notes_to_midi_with_bass(chord_notes)” converts a list of chord notes into MIDI format using a predefined mapping of notes to their MIDI numbers (note_mapping). Additionally, a bass note is added (the lowest note of the chord, reduced by an octave). Returns two lists: chord notes and bass notes.

```
def generate_lead_melody(chords_notes, tempo, repetitions):
    if not chords_notes:
        return []

    lead_notes = []
    time_offset = 0

    for _ in range(repetitions):
        for chord in chords_notes:
            if len(chord) < 3:
                continue

            midi_notes = sorted(chord)
            midi_notes = [note + 12 for note in midi_notes]

            rhythm_pattern = [1, 1, 2]

            note_time = time_offset
            for note, duration in zip(midi_notes, rhythm_pattern):
                lead_notes.append((note, note_time, duration))
                note_time += duration

            time_offset += 4

    return lead_notes
```

The generate_lead_melody(chords_notes, tempo, repetitions) function generates a lead melody based on chord notes. It first checks that chord notes are passed, then creates a lead_notes list where the generated notes will be written. In the loop, the MIDI notes of the chord are first sorted, then transposed up an octave. The rhythmic pattern used is [1, 1, 2], where each note plays in its own time interval, and time_offset keeps track of the overall time offset. The resulting list is returned.

```
def generate_midi_with_bass(notes_of_chord_progression, output_file, repeats, tempo, add_bass, add_lead):
    chord_notes, bass_notes = notes_to_midi_with_bass(notes_of_chord_progression)

    if add_lead:
        lead_melody_per_chord = [generate_lead_melody([ch], tempo, 1) for ch in chord_notes]
    else:
        lead_melody_per_chord = []

    midi = MIDIFile(3 if add_lead else (2 if add_bass else 1))

    track_chords = 0
    track_bass = 1 if add_bass else None
    track_lead = 2 if add_lead else None
    channel = 0
    volume = 100
    lead_volume = 110

    time = 0

    midi.addTempo(track_chords, time, tempo)
    if track_bass is not None:
        midi.addTempo(track_bass, time, tempo)
    if track_lead is not None:
        midi.addTempo(track_lead, time, tempo)

    for z in range(0, repeats * 16, 16):
        for i, chord in enumerate(chord_notes):
            for pitch in chord:
                midi.addNote(track_chords, channel, pitch, i * 4 + z, 4, volume)

    if track_bass is not None:
        for z in range(0, repeats * 16, 16):
            for i, bass in enumerate(bass_notes):
                midi.addNote(track_bass, channel, bass, i * 4 + z, 4, volume)

    if track_lead is not None:
        for repeat_index in range(repeats):
            time_offset = repeat_index * 16

            for chord_index, chord_melody in enumerate(lead_melody_per_chord):
                chord_start_time = time_offset + chord_index * 4

                for lead_note, relative_time, duration in chord_melody:
                    absolute_time = chord_start_time + relative_time
                    midi.addNote(track_lead, channel, lead_note, absolute_time, duration, lead_volume)

    with open(output_file, "wb") as output_f:
        midi.writeFile(output_f)
```

Code part 11.

The generate_midi_with_bass(notes_of_chord_progression, output_file, repeats, tempo, add_bass, add_lead) function creates a MIDI file with chords, bass and lead. First, the chords are converted to MIDI notes with bass notes using notes_to_midi_with_bass. If a lead is added, it is generated for each chord. A MIDIFile is created with the number of tracks depending on the parameters. The first track records the chords, the second the bass, and the third (if enabled) the

lead. In the loop, for each chord, the corresponding notes are added on the timeline. Similar logic is used for the bass. The lead is recorded taking into account the relative time of the notes. The finished MIDI file is saved.

```
def add_drums_to_audio(audio, drum_style, tempo, drum_volume=0.5):
    if drum_style == "No":
        return audio

    if drum_style == "Rock":
        kick = AudioSegment.from_file("rock_kick.wav")
        snare = AudioSegment.from_file("rock_snare.wav")
        hi_hat = AudioSegment.from_file("rock_hihat.wav")
    elif drum_style == "Electronic":
        kick = AudioSegment.from_file("electronic_kick.wav")
        snare = AudioSegment.from_file("electronic_snare.wav")
        hi_hat = AudioSegment.from_file("electronic_hihat.wav")


    kick = kick - (1 - drum_volume) * 30
    snare = snare - (1 - drum_volume) * 30
    hi_hat = hi_hat - (1 - drum_volume) * 30

    beat_duration = 60000 / tempo
    eighth_duration = beat_duration / 2

    one_bar = AudioSegment.silent(duration=4 * beat_duration)

    one_bar = (
        one_bar.overlay(kick, position=0)
        .overlay(snare, position=beat_duration)
        .overlay(kick, position=2 * beat_duration)
        .overlay(snare, position=3 * beat_duration)
    )

    for i in range(0, 8):
        one_bar = one_bar.overlay(hi_hat, position=i * eighth_duration)

    drum_track = one_bar * (len(audio) // len(one_bar) + 1)

    drum_track = drum_track[:len(audio)]

    mixed_audio = audio.overlay(drum_track)
    return mixed_audio
```

Code part 12.

Adds drum tracks to an audio file. Depending on the style (Rock, Electronic), drum sounds (kick, snare, hi-hat) are loaded and a drum pattern is created[4]. The pattern is repeated and superimposed on the main track. The drum volume is adjusted via drum_volume.

```
def midi_to_mp3_with_drums(midi_file, soundfont, mp3_file, drum_style, tempo):
    wav_file = mp3_file.replace('.mp3', '.wav')
    os.system(f'fluidsynth -ni {soundfont} {midi_file} -F {wav_file} -r 44100')
    audio = AudioSegment.from_wav(wav_file)
    audio_with_drums = add_drums_to_audio(audio, drum_style, tempo)
    audio_with_drums.export(mp3_file, format='mp3')

    os.remove(wav_file)

def soundfont_choose(sf_name):
    if sf_name == 'Piano':
        sf = 'GeneralUser_GS_v1.471.sf2'
    elif sf_name == 'Marimba':
        sf = 'marimba-deadstroke.sf2'
    elif sf_name == 'Old video games':
        sf = 'PICO-8_1.1.2.sf2'
    return sf
```

Code part 13.

“midi_to_mp3_with_drums(midi_file, soundfont, mp3_file, drum_style, tempo)” converts a MIDI file to MP3 using the specified soundfont and adds drums. First, the MIDI is converted to WAV using fluidsynth, then the drums are superimposed on the audio file and the result is saved as MP3.

The “soundfont_choose” function with the “sf_name” string parameter selects a soundfont file relative to the given string. The program is planned to have a synthesizer selection button. The files were previously added.

The following synthesizers were selected: piano (GeneralUser GS), marimba (Marimba Deadstroke), old video games (PICO 8_1.1.2), the attribution to which will be indicated in the program itself.[5]

```
def play_mp3():
    generate_mp3()
    if last_generated_mp3 and os.path.exists(last_generated_mp3):
        pygame.mixer.music.load(last_generated_mp3)
        pygame.mixer.music.play()
    else:
        messagebox.showerror("Error", "No MP3 file available to play.")

def stop_mp3():
    pygame.mixer.music.stop()

pygame.mixer.init()
last_generated_chords = []
root = Tk()
root.title("MakeTheMusic")
root.geometry("700x700")

bg_image = None
bg_label = None

def set_background(image_path):
    global bg_image, bg_label
    if bg_label is not None:
        bg_label.destroy()
    try:
        bg_image = PhotoImage(file=image_path)
        bg_label = Label(root, image=bg_image)
        bg_label.place(x=0, y=0, relwidth=1, relheight=1)
        bg_label.lower(header)
    except TclError:
        messagebox.showerror("Error", f"Image file not found: {image_path}")
```

Code part 14.

The play_mp3() and stop_mp3() functions are responsible for playing and stopping the MP3 file using pygame.mixer.music. If the file exists, it is loaded and played. The set_background(image_path) function sets the background image in the window. If the image is not found, an error message is displayed.

```
header = LabelFrame(root, text="Sound", padx=10, pady=10)
header.pack(fill="x", padx=10, pady=10)

input_wrap = Frame(header)
options_major_minor = ["happy", "sad"]
value_inside = StringVar(root)
value_inside.set(options_major_minor[0])
dropdown_minor_major = ttk.Combobox(input_wrap, values=options_major_minor, state="readonly", textvariable=value_inside)
dropdown_minor_major.pack(side=LEFT)

options_chord = ['Any', 'C', 'Db', 'D', 'Eb', 'E', 'F', 'Gb', 'G', 'Ab', 'A', 'Bb', 'B', 'Cm', 'Dbm', 'Dm', 'Ebm', 'Em',
value_inside_n = StringVar(root)
value_inside_n.set(options_chord[0])
dropdown_chord = ttk.Combobox(input_wrap, values=options_chord, state="readonly", textvariable=value_inside_n)
dropdown_chord.pack(side=LEFT)

input_wrap.pack()
label_wrapper = Frame(header)
Label(label_wrapper, text="Mood").pack(side=LEFT)
Label(label_wrapper, text="First chord").pack(side=LEFT)
label_wrapper.pack()

res_field_text = StringVar()
res_field = Label(root, textvariable=res_field_text)

output_settings = LabelFrame(root, text="Output settings:")
output_settings.pack_forget()

Label(output_settings, text="BPM:").grid(row=0, column=0, sticky=W)
tempo_entry = Entry(output_settings)
tempo_entry.insert(0, "120")
tempo_entry.grid(row=0, column=1)

Label(output_settings, text="Repetitions:").grid(row=1, column=0, sticky=W)
repetitions_entry = Entry(output_settings)
repetitions_entry.insert(0, "1")
repetitions_entry.grid(row=1, column=1)

synth_var_custom = StringVar(root)
synth_var_custom.set("")

def on_synthesizer_selected(selected_option):
    if selected_option == "Other":
        filename = filedialog.askopenfilename(filetypes=[("SoundFont files", "*.sf2")])
        if filename:
            synth_var_custom.set(filename)
    else:
        synth_var_custom.set("")
```

Code part 15.

```
synth_var = StringVar(root)
synth_var.set("Piano")
Label(output_settings, text="Choose the synthesizer for MP3").grid(row=2, column=0, sticky=W)
synth_options = ttk.Combobox(output_settings, values=["Piano", "Old video games", "Marimba", "Other"], state="readonly", textvariable=synth_var)
synth_options.grid(row=2, column=1)
synth_options.bind("<<ComboboxSelected>>", lambda e: on_synthesizer_selected(synth_var.get()))

drum_style_var = StringVar(root)
drum_style_var.set("No")
Label(output_settings, text="Add drums:").grid(row=3, column=0, sticky=W)
drum_style_options = ttk.Combobox(output_settings, values=["No", "Rock", "Electronic"], state="readonly", textvariable=drum_style_var)
drum_style_options.grid(row=3, column=1)

bass_line_var = BooleanVar()
Checkbutton(output_settings, text="Add the bass line", variable=bass_line_var).grid(row=4, columnspan=2, sticky=W)
lead_melody_var = BooleanVar()
Checkbutton(output_settings, text="Add a lead melody", variable=lead_melody_var).grid(row=5, columnspan=2, sticky=W)

def on_generate():
    global last_generated_chords
    mood = value_inside.get()
    selected_start_chord = value_inside_n.get()

    if selected_start_chord == 'Any':
        selected_start_chord = random.choice(options_chord[1:])

    new_chords = generate_chords(mood, selected_start_chord)

    while new_chords == last_generated_chords:
        new_chords = generate_chords(mood, selected_start_chord)

    last_generated_chords = new_chords
    res_field_text.set(' '.join(new_chords))
    output_settings.pack(pady=10)
    generate_mp3()
    play_button.pack(side=LEFT, padx=10)
    stop_button.pack(side=RIGHT, padx=10)

def on_generate_midi():
    tempo = int(tempo_entry.get())
    repetitions = int(repetitions_entry.get())
    notes = chords_to_notes(last_generated_chords)
    output_file = filedialog.asksaveasfilename(defaultextension=".mid", filetypes=[("MIDI files", "*.mid")])
    if not output_file:
        return
    add_bass = bass_line_var.get()
    add_lead = lead_melody_var.get()
    generate_midi_with_bass(notes, output_file, repetitions, tempo, add_bass, add_lead)
```

Code part 16.

```
def generate_mp3():
    global last_generated_mp3

    if synth_var.get() == "Other":
        soundfont = synth_var_custom.get()
    else:
        soundfont = soundfont_choose(synth_var.get())

    repetitions = int(repetitions_entry.get())
    tempo = int(tempo_entry.get())
    drum_style = drum_style_var.get()

    chords_notes = chords_to_notes(last_generated_chords)

    add_bass = bass_line_var.get()
    add_lead = lead_melody_var.get()

    with tempfile.NamedTemporaryFile(suffix=".mid", delete=False) as temp_midi:
        generate_midi_with_bass(chords_notes, temp_midi.name, repetitions, tempo, add_bass, add_lead)

        temp_mp3 = tempfile.NamedTemporaryFile(suffix=".mp3", delete=False)
        last_generated_mp3 = temp_mp3.name

        midi_to_mp3_with_drums(temp_midi.name, soundfont, last_generated_mp3, drum_style, tempo)
        os.remove(temp_midi.name)

def save_mp3_file():
    if last_generated_mp3:
        save_path = filedialog.asksaveasfilename(defaultextension=".mp3", filetypes=[("MP3 files", "*.mp3")])
        if save_path:
            os.rename(last_generated_mp3, save_path)
            last_generated_mp3 = save_path

set_background("background.png")
```

Code part 17.

```
control_frame = Frame(root)
control_frame.pack()

play_button = Button(control_frame, text="▶ Play", command=play_mp3)
stop_button = Button(control_frame, text="■ Stop", command=stop_mp3)

play_button.pack_forget()
stop_button.pack_forget()

res_field.pack(padx=50, pady=5)
Button(output_settings, text="Save MIDI", command=on_generate_midi).grid(row=6, column=0, pady=10)
Button(output_settings, text="Save MP3", command=save_mp3_file).grid(row=6, column=1, pady=10)

bottom_frame = Frame(root)
bottom_frame.pack(side="bottom", fill="x", pady=10)

def open_window_with_text(file_path):
    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            text = file.read()
        new_window = Toplevel(root)
        new_window.title(file_path)

        text_widget = Text(new_window, wrap="word", width=60, height=20)
        text_widget.insert("1.0", text)
        text_widget.config(state="disabled")
        text_widget.pack(padx=10, pady=10, expand=True, fill="both")

        scrollbar = Scrollbar(new_window, command=text_widget.yview)
        scrollbar.pack(side="right", fill="y")
        text_widget.config(yscrollcommand=scrollbar.set)

    except FileNotFoundError:
        messagebox.showerror("Error", f"File not found: {file_path}")

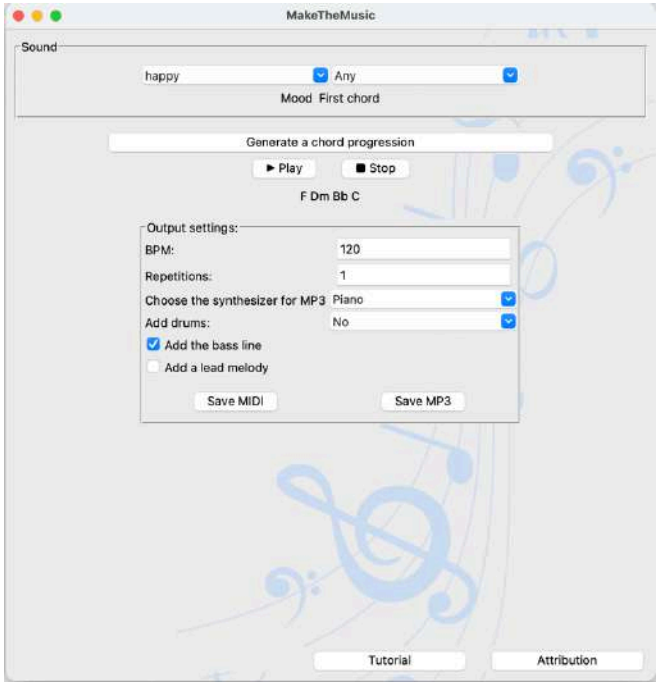

attribution_button = Button(bottom_frame, text="Attribution", width=15, command=lambda: open_window_with_text("textfiles/atribution.txt"))
attribution_button.pack(side="right", padx=10)

tutorial_button = Button(bottom_frame, text="Tutorial", width=15, command=lambda: open_window_with_text("textfiles/tutorial.txt"))
tutorial_button.pack(side="right", padx=10)

root.mainloop()
```

Code part 18.

The on_generate() function is responsible for generating the chord progression. First, the mood and the first chord are selected, then a new sequence is generated. If it matches the previous one, a new generation is performed. The result is displayed and the UI elements for saving and playing are enabled. The on_generate_midi() function saves the MIDI file. The tempo, number of repetitions, chord notes are obtained, then generate_midi_with_bass is called to save the file. The generate_mp3() function creates an MP3 file based on the selected SoundFont, number of repeats, tempo, drum style, and whether or not a bass or lead melody is present. A temporary MIDI file is generated, converted to MP3, and saved. The save_mp3_file() function allows the user to select where to save the generated MP3 file. The open_window_with_text(file_path) function opens a new window and loads a text file, such as attribution or tutorial material. If the file is not found, an error is displayed. The code uses tkinter to create the interface, including buttons, drop-down lists, and text fields. The background image is loaded via set_background. Chords are generated by pressing the button, after which playback and saving controls become available. The Generate a chord progression, Save MIDI, Save MP3, and Attribution buttons are bound to the appropriate functions. As a result, the program allows you to select the key, chord style, create a musical sequence with the ability to add drums, save it to MIDI or MP3 and play it.



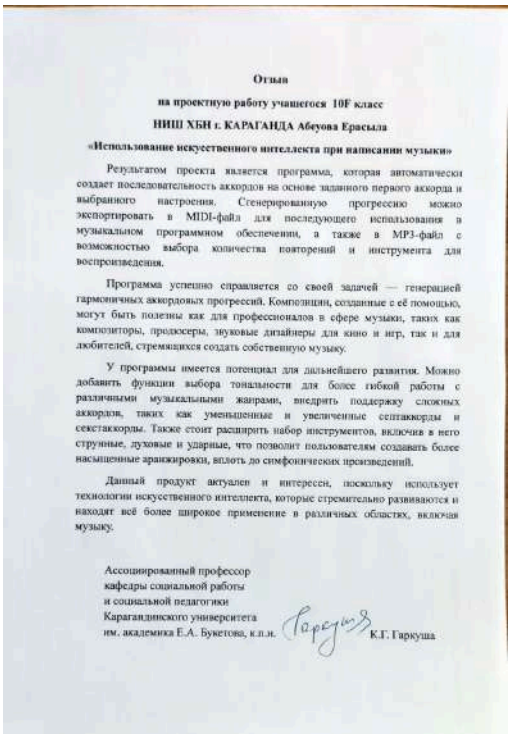
Picture 10. The app.

Ultimately, in the app, the user first selects the mood and the first chord. Then, by pressing a button, a chord progression is generated. MIDI or MP3 formats can be selected for output. There is an option to add drums, change the synthesizer, add a bass line or a lead melody. The user can play the generated MP3-file. Also, the user can refer to the instructions and attribution.

Conclusion

In conclusion, I would like to note that the step-by-step implementation of the initial tasks led to more efficient work on the implementation of my idea. I was able to get acquainted with music theory, namely the rules of harmony of chord sequences. This theory was used when creating training files for the AI model. The basic concept of neural networks introduced me to the work of AI. A review of analogs allowed me to identify their shortcomings, which, as far as possible, I tried to take into account when creating the program. When choosing a programming language and libraries, as well as when writing the code, my work plan (code work schemes) helped me, which I relied on. The choice of the Python programming language turned out to be favorable due to the simplicity and "flexibility" of the language. The main advantage of Python was the presence of libraries for training and creating artificial intelligence, while in other languages, libraries are either not so convenient or do not exist. These libraries and others (for working with chords, MIDI and their playback, matrices, graphical interface) were studied and used during the project. Detailed work was carried out on additional codes and the main program code of my program.

My final product has advantages that were not observed in some analogues. The first advantage is export to MIDI and MP3. The second is the ability to customize the synthesizer that plays the progression in the audio track. The use of a trained AI model in the program can be called a unique approach to the implementation of my idea. It is important to consider that neural networks saved time on the implementation of more in-depth musical theory in the code. It is worth considering that the product requires improvements. For instance, other instruments could be added for creating a symphony. In the near future, it is definitely planned to obtain copyright for my program.



Отзыв

на проектную работу учащегося 10F класс

НИШ ХБН г. КАРАГАНДА Абеуова Ерасыла

«Использование искусственного интеллекта при написании музыки»

Результатом проекта является программа, которая автоматически создает последовательность аккордов на основе заданного первого аккорда и выбранного настроения. Сгенерированную прогрессию можно экспортировать в MIDI-файл для последующего использования в музыкальном программном обеспечении, а также в MP3-файл с возможностью выбора количества повторений и инструмента для воспроизведения.

Программа успешно справляется со своей задачей — генерацией гармоничных аккордовых прогрессий. Композиции, созданные с её помощью, могут быть полезны как для профессионалов в сфере музыки, таких как композиторы, продюсеры, звуковые дизайнеры для кино и игр, так и для любителей, стремящихся создать собственную музыку.

У программы имеется потенциал для дальнейшего развития. Можно добавить функции выбора тональности для более гибкой работы с различными музыкальными жанрами, внедрить поддержку сложных аккордов, таких как уменьшенные и увеличенные септаккорды и секстаккорды. Также стоит расширить набор инструментов, включив в него струнные, духовые и ударные, что позволит пользователям создавать более насыщенные аранжировки, вплоть до симфонических произведений.

Данный продукт актуален и интересен, поскольку использует технологии искусственного интеллекта, которые стремительно развиваются и находят всё более широкое применение в различных областях, включая музыку.

Ассоциированный профессор
кафедры социальной работы
и социальной педагогики
Карагандинского университета
им. академика Е.А. Букетова, к.п.н. К.Г. Гаркуша

There is a review (in Russian, the image is given on the left) from an associate professor of the Department of Social Work and Social Pedagogy of the Karaganda University named after Buketov K.G. Garkusha, who also works as a music teacher. She claims that the program copes with its task, has potential for further development, is relevant and interesting due to the use of artificial intelligence.

The mentor of this project (Ustinova Lyudmila Valeryevna, teacher of informatics) has provided feedback: “It is especially worth noting the high level of independence of the author mastering complex topics. Yerassyl independently studied the basics of musical theories, principles of working with neural networks and mastered programming in Python. The project demonstrates an excellent combination of theoretical knowledge and practical skills. The author showed great independence in studying new topics and solving complex problems, which is an excellent example of his scientific and creative potential. This project is not only has practical value, but also serves as an example of how artificial intelligence can be integrated into creative processes”.

References

[1] – *IFPI Global Music Report 2024, International Federation of the Phonographic Industry, 2024, Page 11*
[2] – *First recording of computer-generated music – created by Alan Turing – restored, The Guardian, Agence France-Presse in Wellington,* 26/09/2016
[3] – *2023 Stack Overflow Developer Survey, Stack Overflow,* 2023
[4] – Rock drums - Emerson Vernon (Alternative Rock Loops | Producer Pack) https://emersonvernon.com/store/p/alternative-rock-loops-producer-pack
Electronic drums - Trisamples The Sample Set Vol.2 https://trisamples.com/the-sample-set-vol-2/#google_vignette
[5] – "Piano" (GeneralUser GS 1.471) - https://schristiancollins.com/generaluser.php
"Old video games" (PICO-8_1.1.2, создано Packbat & Bikibird) - https://musical-artifacts.com/artifacts/3312
"Marimba" (marimba-deadstroke-ff-sf2) - https://musical-artifacts.com/artifacts/416